- **Internship Name : Data Analysis**
- **Task Name: Task 5: Learn NumPy Array Creation, Indexing, and Slicing; Compare Array Operations with Plain Python Lists**
- **Your Name : Shubhangi Naresh Khonde**
- **Internship Organization :**
- **Date:**

1.Objective:

The objective of this task was to learn the basics of the NumPy library in Python. The task focused on creating NumPy arrays, accessing elements using indexing, extracting elements using slicing, performing mathematical operations on arrays, and comparing NumPy arrays with standard Python lists.

2.Introduction:

NumPy (Numerical Python) is an important Python library used for numerical computing and data analysis. It provides a powerful array object called ndarray, which allows efficient storage and processing of numerical data.

In this task, NumPy arrays were created and different operations such as indexing, slicing, addition, subtraction, multiplication, and division were performed. The behavior of NumPy arrays was also compared with normal Python lists to understand the advantages of using NumPy for numerical operations.

Tools Used

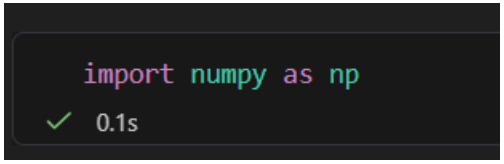
- Python
- Jupyter Notebook
- NumPy Library

3.Concepts Covered

1. Importing NumPy
2. Creating 1D NumPy arrays
3. Creating 2D NumPy arrays
4. Checking array dimensions
5. Array indexing
6. Array slicing
7. Mathematical operations on arrays
8. Operations between two arrays
9. Comparison between Python Lists and NumPy Array

4.Code / Implementation:

1.Importing NumPy:



The NumPy library was imported using the alias `np`.

2.Creating a 1D Array:

```
##Create a NumPy Array
```

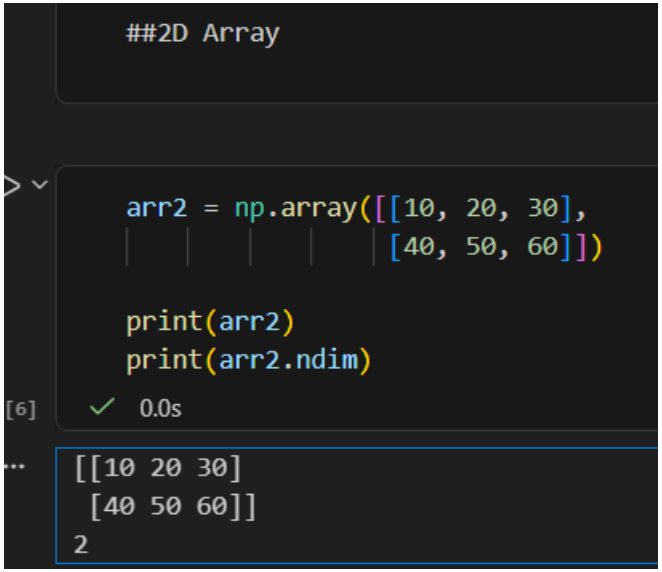
```
import numpy as np
numbers = np.array([10,20,30,40,50])
print(numbers)
print(type(numbers))
```

✓ 0.0s

```
[10 20 30 40 50]
<class 'numpy.ndarray'>
```

3. Creating a 2D Array:

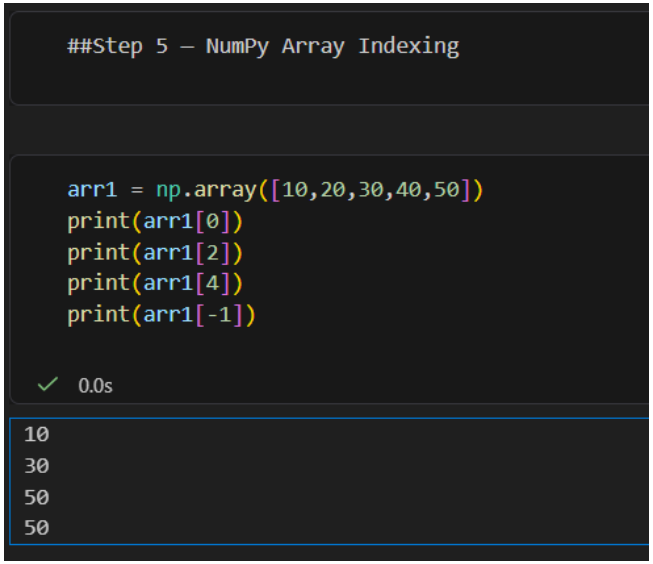
4. Checking Dimensions:



The array contains 2 rows and 3 columns.

The ndim attribute returns the number of dimensions of the array.

5. Array Indexing:



NumPy indexing starts from index 0. Negative indexing can be used to access elements from the end of the array.

For a 2D array:

```
##2D Array Indexing
```

```
arr2 = np.array([[10, 20, 30],
                 [40, 50, 60]])
print(arr2[0,1])
print(arr2[1,1])
```

[8] ✓ 0.0s

```
20
50
```

6. Array Slicing

```
##Step 6 – NumPy Array Slicing
```

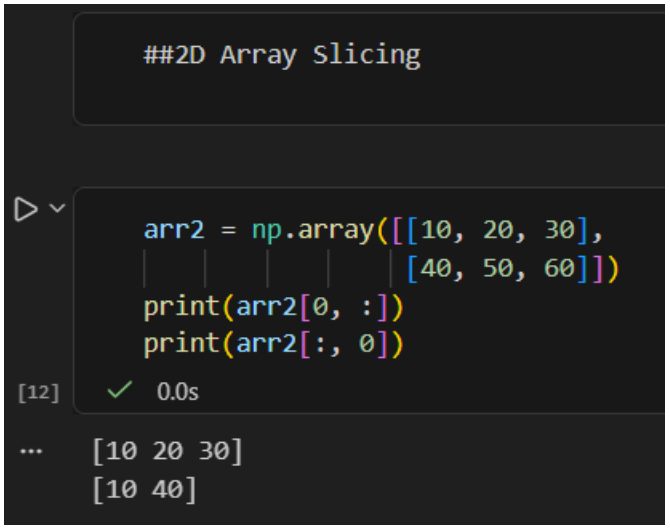
```
arr1 = np.array([10,20,30,40,50])
print(arr1[1:4])
print(arr1[:3])
print(arr1[2:])
print(arr1[::2])
```

] ✓ 0.0s

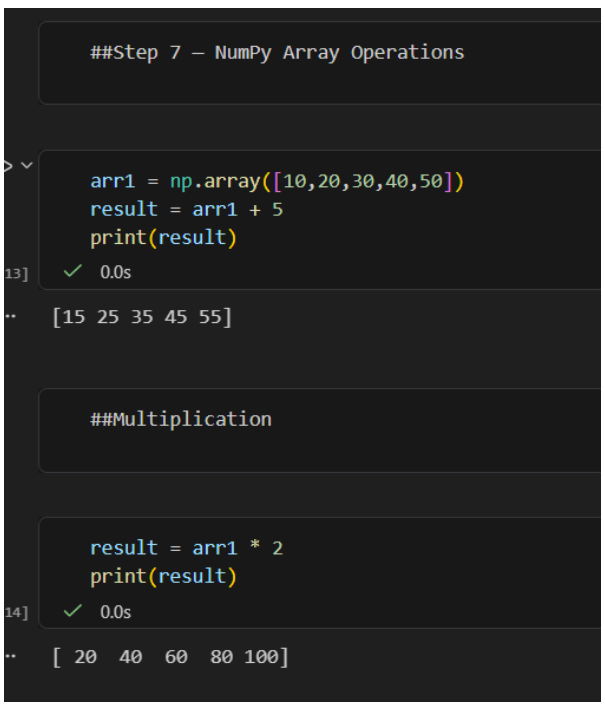
```
[20 30 40]
[10 20 30]
[30 40 50]
[10 30 50]
```

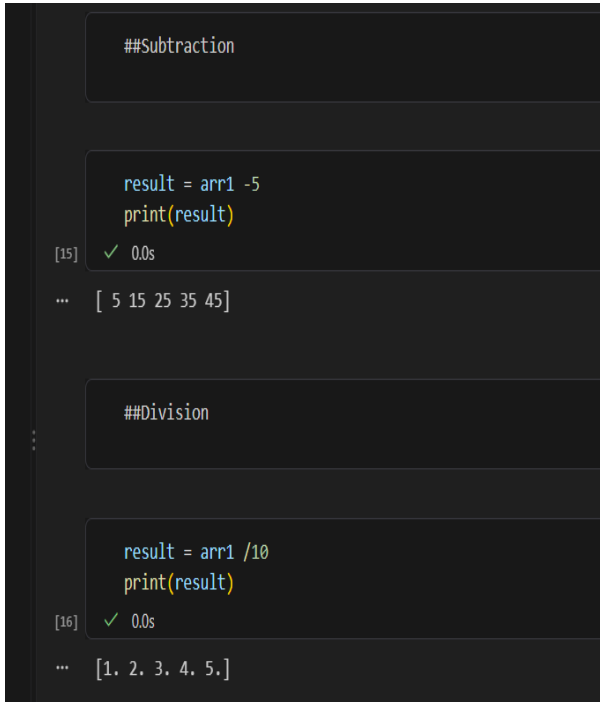
Slicing is used to extract a specific portion of an array.

For a 2D array:

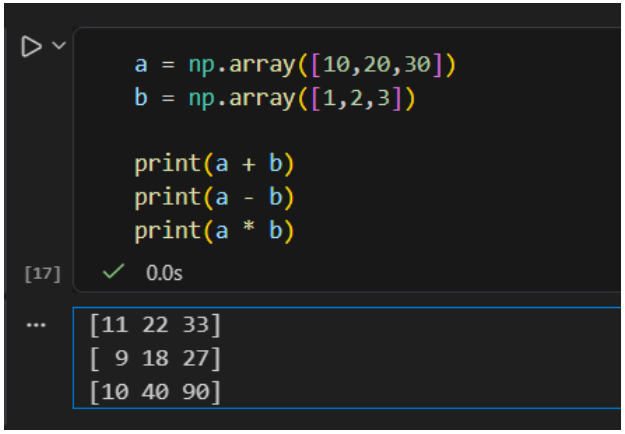


7. Mathematical Operations on NumPy Arrays





8. Operations Between Two Arrays



The operations are performed element-by-element.

10. Python List vs NumPy Array

```
##Step 9 – Python List vs NumPy Array


list1 = [10,20,30,40,50]
print(list1)
print(type(list1))
print(list1 * 2)
```

✓ 0.0s

```
[10, 20, 30, 40, 50]
<class 'list'>
[10, 20, 30, 40, 50, 10, 20, 30, 40, 50]
```

NumPy performs the operation on each element of the array.

5.Results:

The following results were achieved during the task:

- Successfully imported the NumPy library.
- Created and displayed 1D and 2D NumPy arrays.
- Successfully accessed array elements using indexing.
- Extracted selected elements using slicing.
- Performed arithmetic operations on NumPy arrays.
- Performed element-wise operations between two arrays.
- Compared the behavior of Python lists and NumPy arrays.

6.Key Learning:

Through this task, I learned that NumPy provides an efficient and convenient way to work with numerical data in Python. I understood how to create arrays, access specific elements, extract portions of arrays, and perform mathematical operations.
I also learned the difference between Python lists and NumPy arrays. NumPy arrays support direct element-wise mathematical operations, which makes them highly useful for numerical calculations and data analysis.

7.Conclusion:

The NumPy array creation, indexing, slicing, and operations task was successfully completed. I gained practical knowledge of NumPy arrays and understood how they differ from standard Python lists.
This task provided a strong foundation for further learning in Python-based data analysis. The concepts learned here will be useful for working with libraries such as Pandas and for performing data manipulation and numerical analysis in future tasks.